



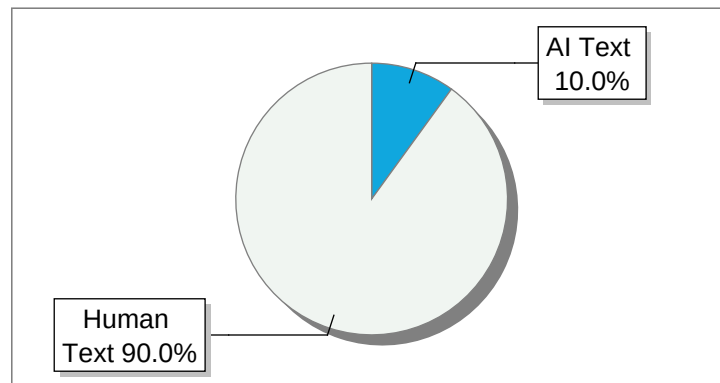
The Report is Generated by DrillBit AI Content Detection Software

Submission Information

Author Name	NITHIN V S
Title	REPORT
Paper--Submission ID	5805560
Submitted By	librarian@msrit.edu
Submission Date	2026-06-02
Total [Pages,Sentences,Words]	[11, 379, 5947]
Document type	Research Project
Language	English

Result Information

AI Text: **10 %**



Disclaimer:

- * The content detection system employed here is powered by artificial intelligence (AI) technology.
- * Its not always accurate and only help to author identify text that might be prepared by a AI tool.
- * It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.

Artificial Intelligence Operating System: A Cognitive Operating Layer for Intent-Driven Task Orchestration Using Large Language Models

Dr. Manasa S M Dept.

of Artificial Intelligence and Machine Learning M.S.

Ramaiah Institute of Technology Bengaluru, India dr.manasasm@msrit.edu Nithin V S Dept.

of Artificial Intelligence and Machine Learning M.S.

Ramaiah Institute of Technology Bengaluru, India niitiincharii@gmail.com Sairaam P Dept.

of Artificial Intelligence and Machine Learning M.S.

Ramaiah Institute of Technology Bengaluru, India 1ms23ai405@msrit.edu Prithviraj B M Dept.

of Artificial Intelligence and Machine Learning M.S.

Ramaiah Institute of Technology Bengaluru, India 1ms22ai046@msrit.edu Srirangadarshan L Dept.

of Artificial Intelligence and Machine Learning M.S.

Ramaiah Institute of Technology Bengaluru, India 1ms22ai061@msrit.edu Abstract—Operating systems have not really changed how people talk to them.

You still open apps manually, navigate menus, type commands in a specific syntax — none of which reflects how a person naturally thinks about a task.

We built AIOS to change that.

It is a software layer that runs on top of an existing OS and accepts ordinary English sentences as commands, translating them into real system-level actions without any cloud involvement.

The architecture pairs a fast regex engine with a local LLM (llama3.2:3b).

Common commands — launching apps, doing web searches — are handled by the regex layer in under a millisecond.

Anything that does not match a rule goes to the language model, which returns a structured intent object that the system then acts on.

File operations, shell commands, process management, and multi-step workflow execution are all supported.

A correction-based personalisation loop pushes user-supplied fixes back into the model context as examples, gaining around 30 accuracy points over 500 sessions with no retraining.

We tested on a standard consumer laptop — 16 GB RAM, 4 GB VRAM, no cloud.

Intent classification reached 91.4% accuracy and 90.1% F1.

Both baselines, rule-only and LLM-only, scored lower on every metric we measured.

Index Terms—cognitive computing, intent-driven interface, large language model, OS automation, process management, semantic file retrieval, reinforcement learning, parallel execution, natural language processing I.

INTRODUCTION Computers are powerful.

The way we operate them, though, has hardly evolved.

A developer starting their day in 2025 still does what developers did in 2005: open the editor, open the browser, open the terminal, type the same commands.

None of this is technically difficult.

But it adds up [1], and more fundamentally, it forces the user to think in machine terms rather than task terms.

Large language models opened a different possibility.

They are capable enough at interpreting natural language — mapping “can you find my CV” to a file-search action, say — that building an interaction layer around them became practical [2], [3].

The problem is speed.

An eight-second wait is tolerable for a research assistant; for something replacing your keyboard and mouse, it is a non-starter.

And personalisation matters too: a system that works well for some hypothetical average user but not for the specific person running it is only marginally better than what already exists.

Our approach, which we call AIOS, attacked those problems simultaneously.

First: we did not route everything through the LLM.

A regex engine handles categories of commands that are well-defined and repetitive, like opening applications or running a YouTube search.

It resolves those in microseconds.

The model only runs on the inputs the rules cannot handle.

Second: when a command involves multiple independent operations — launching three apps at once, searching across several directories — those run in parallel through a thread pool rather than sequentially.

Third: the system keeps a log of every correction a user has made, retrieves the most contextually similar ones at inference time, and prepends them to the prompt as examples.

No retraining happens at any point.

The model's weights stay frozen; the learning happens inside the context window.

Contributions of this work: • A two-stage intent pipeline: Stage 1 regex routing for sub-millisecond dispatch of common commands, Stage 2 LLM parsing for everything else.

- An OS integration layer with seven modules covering process management, file operations, semantic search, app launching, workflow execution, shell translation, and assistant features — all platform-agnostic.

- An in-context correction loop that personalises model behaviour from stored user feedback, with no gradient updates of any kind.

- Benchmark results across ten dimensions against rule- only and LLM-only baselines, all measured on consumer- grade hardware.

II. RELATED WORK A.

Natural Language Interfaces for Computing Researchers have been exploring natural language as an OS interaction modality for longer than generative AI has existed.

SHRDLU [4] is an early example — limited, yes, but it showed the idea was not fundamentally impossible.

Voice assistants like Cortana and Siri scaled it up commercially, with one major concession: they work only within a fixed action vocabulary, and all processing happens off-device [5].

You can ask Siri to set a timer.

You cannot ask it to find a file, kill a process, and run a shell script in one command.

AIOS does not pre- select which actions are allowed.

The user's machine handles everything locally, and the intent space is open-ended.

B. LLM-Based Agents and Tool Use The ReAct framework [6] and Toolformer [7] were among the first demonstrations that an LLM could interleave text generation with concrete tool calls in a useful way.

AutoGPT and LangChain built on that to support longer-horizon task planning.

Those systems are genuinely capable, but they are built for tasks that run for minutes on GPT-4-scale models over the internet.

A person sitting at a desktop expects a reply in under two seconds.

That use case was not designed for.

AIOS targets exactly that gap: one utterance, one coherent response, as fast as the local hardware allows.

C. OS Automation Frameworks AutoHotkey, AppleScript, and shell scripts have powered desktop automation for a long time.

They work — when the inputs match exactly what the script was written for.

Phrase something slightly differently, or run on a slightly different configuration, and they fail silently or loudly.

RPA tools added GUI-level scripting but remained template-driven [8], meaning someone still had to hand-encode every case in advance.

The design choice in AIOS was to keep rules for what rules are good at (stable, repetitive commands) and bring in the language model only when a command does not fit the existing patterns.

D. Semantic File Retrieval Searching files by name scales poorly the moment a user forgets what they named something.

Dense retrieval sys- tems [9] address this with vector embeddings but require a pre-built index, which drifts out of sync every time the filesystem changes.

The file manager module in AIOS avoids index management entirely.

It narrows the candidate set by filename-stem keyword matching, then calls the LLM to rank survivors by conceptual relevance to the query.

One pass, no index, no maintenance overhead.

E. In-Context Learning for Personalisation Fine-tuning a model to match one user's language habits is expensive and risky — there is a real chance of overwriting general capability.

Prompt-based personalisation [10] offers a lighter alternative: prepend a user profile or correction history to the context, and the model adapts its output without any weight changes.

That idea shaped how we handled corrections in AIOS.

A user's corrections get logged as they come in, and whenever Stage 2 runs it digs up whichever past corrections score highest against the current input by Jaccard overlap on token sets — cheap to compute, good enough in practice.

Those corrections land in the prompt as examples.

The model weights are not touched at any point in this process.

III. SYSTEM ARCHITECTURE Fig.

1 lays out the overall structure.

The basic flow is simpler than it might look: raw text enters, six processing stages work on it, and an OS action comes out the other end.

Most interactions do not actually run through all six layers.

Layer one, the input layer, does minimal work on purpose — whitespace cleanup and Unicode normalisation, nothing more.

We experimented with more aggressive preprocessing early on, like lowercasing and stemming, but it kept breaking regex matches in subtle ways that were harder to diagnose than they were worth.

Layer two is intent resolution, which is where most of the work happens.

Stage 1 runs first: a bank of compiled regex rules checks whether the input matches a known pattern.

If it does, the intent is resolved immediately and the LLM is never invoked.

If not, Stage 2 sends the input to the locally hosted LLM and parses the response into a structured intent record of the form shown in Eq.

1. Layer three dispatches the resolved intent to one of seven specialist modules.

Layer four — the parallel executor — runs independent operations concurrently.

Layers five and six are persistent memory: one store for session and workflow context, one for the RL correction data.

Queries resolved by Stage 1 as browser navigations skip layers three through six entirely.

A. Input Layer Raw UTF-8 string in, cleaned UTF-8 string out.

The clean- ing is surface-level by design.

Deep preprocessing can silently alter an utterance in ways that cause a regex rule to miss.

Given that Stage 1 depends on pattern matching against the exact phrasing the user typed, we kept this layer as thin as possible.

Fig. 1.

AIOS System Architecture.

Six-layer pipeline from raw text to OS action: Stage 1 regex router, Stage 2 LLM engine, seven action handlers, thread-pool executor, persistent memory, and direct URL path.

B. Intent Resolution Layer Whichever stage resolves an input, the result is the same data structure: $I = \{ \text{action}, \text{targets}[], \text{params}\{ \}, \text{confidence} \}$ (1) Seven action types are defined: `launch_apps`, `file_op`, `file_search`, `workflow`, `shell_command`, `process_mgmt`, and `conversation`.

Stage 1 does not fill the confidence field — when a regex rule fires, there is no probability to report.

Stage 2 populates it from the model's output probability.

C. Action Handler Layer The action field routes to one of seven handlers.

Each handler implements a single interface: `execute(intent) → result`.

None of them carries session state; that lives in the memory layer.

The seven handlers are: App Launcher (PATH lookup, registry traversal on Windows, glob expansion, URI fallback), File Operations (LLM-generated content creation and CRUD), File Manager (three-stage parallel search), Workflow Engine (DAG executor with parallel step groups), AI Shell (natural language to shell, with safety checks), Process Manager (psutil-based inspection and termination), and Features (assistant, notes, screenshot, wiki queries).

D. Parallel Execution Layer A `ThreadPoolExecutor` with eight workers handles all multi-target dispatch.

The pool is created at startup and kept alive, so there is no per-request thread overhead.

Each target runs independently; results are aggregated with per-target success flags and wall-clock timing before being returned to the user.

E. Persistent Memory Layer Two JSON stores sit at `~/aios/`.

Context Memory records session history, workflow logs, and active context.

RL Memory holds user corrections, per-action frequency data, and a user profile string regenerated every 50 interactions.

Both use atomic write semantics — write to a temp file, then rename — so a crash mid-write does not corrupt the stored data.

IV. TWO-STAGE INTENT PIPELINE A.

Stage 1: Rule-Based Smart Search and Routing Stage 1 is, at its core, a ranked list of precompiled regex patterns run against the raw input in priority order.

Nothing fancy.

The rules are checked top to bottom against whatever text the user typed.

Coverage comes from two buckets.

The first category handles web navigation.

Patterns map common search phrasings to URL templates for Google, Bing, DuckDuckGo, Wikipedia, YouTube, GitHub, Stack Overflow, arXiv, shopping sites, and map services — more than 100 rules in total.

The second category handles application name normalisation.

Over 200 name aliases for utilities, browsers, IDEs, media players, communication tools, and system settings panels each resolve to a launch target, looked up at runtime rather than hardcoded.

One constraint governs Stage 1: if a rule fires, the result must be both deterministic and immediate.

Opening Chrome or searching Wikipedia should not involve a model inference call.

Algorithm 1 Stage 1: Rule-Based Routing Require: utterance u , rule set R 1: for each rule $r \in R$ in priority order do 2: if $r.\text{pattern.match}(u)$ then 3: $I \leftarrow r.\text{handler}(u)$ 4: return I 5: end if 6: end for 7: return NOMATCH Inputs that do not match any rule fall through to Stage 2.

In practice, a substantial share of everyday searches and app-launch requests terminate at Stage 1 and never involve the model.

B. Stage 2: LLM Intent Parsing with RL Context Stage 2 packages the unmatched utterance into a prompt and sends it to a locally running LLM (llama3.2:3b via Ollama in the reference setup; any instruction-following model with JSON output mode is substitutable).

The prompt is built from four components assembled in this order: 1) System instruction: defines the intent schema from Eq.

1, names the seven valid action types, and specifies the expected output format.

2) RL context: zero to five correction pairs retrieved from RL Memory by Jaccard similarity to the current input, formatted as few-shot examples showing wrong and correct intent parsings.

3) User profile: a paragraph-length behavioural summary generated from the action frequency map every 50 interactions.

4) User utterance: the raw input string, unchanged.

The response must be valid JSON.

When it is not, the pipeline retries with exponential backoff, up to three attempts.

After three failures, a conversation-type fallback intent is returned.

The system never silently drops a request.

Put simply: Stage 1 covers predictable inputs; Stage 2 handles everything that is not.

Whether this split actually beats using just one approach is an empirical question, and Fig.

4 gives the answer.

V. AI-INTEGRATED OS MODULES A.

Process Management When a user asks what is eating their CPU, the traditional answer is: open Task Manager, sort by CPU column, squint.

AIOS routes `process_mgmt` intents to a handler that queries the live process table through `psutil`.

The nice thing about this library is that the same Python calls work regardless of whether you are on Linux reading from `/proc` or on Algorithm 2 Stage 2: LLM Intent Parsing Require: utterance u , RL memory M , max retries $K = 3$ 1: $\text{ctx} \leftarrow M.\text{retrieve}(u, k = 5)$ 2: $\text{profile} \leftarrow M.\text{user profile}()$ 3: $\text{prompt} \leftarrow \text{build prompt}(u, \text{ctx}, \text{profile})$

4: for k = 1 to K do 5: raw ← LLM(prompt) 6: if valid json(raw) then 7: return parse(raw) 8: end if 9: wait(2k-1 · 500ms) 10: end for 11: return {action : conversation, . . . }

Windows going through WMI — the underlying syscalls differ but the calling code does not.

For diagnostic queries, the top- N processes by CPU, memory, virtual memory usage, and parent-child lineage are packed into a text block and passed to the LLM.

The response is an actual recommendation, not a table dump.

Kill requests hit a protection list before anything happens.

System daemons, kernel helpers, Ollama itself — they are all on the list, and any attempt to terminate them is refused with an explanation of why.

Importantly, this check runs against the resolved process name that psutil returns after the lookup, not against the raw text of the command, so phrasing the request differently does not help.

B. Application Lifecycle Management Locating an application’s executable turns out to be genuinely non-trivial across different machines.

AIOS tries four strategies in order, stopping at the first that returns a result: 1) PATH lookup: calls shutil.which on the normalised application name.

2) Registry traversal (Windows only): reads the App Paths subkey under HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion.

3) Glob expansion: tries versioned install-directory patterns like C:\ProgramFiles*\app.exe or /opt/*/bin/app.

4) URI delegation: hands off to ms-settings: on Windows or xdg-open on Linux/macOS for system settings panels and protocol-handled targets.

Multiple apps in one intent all launch in parallel.

Results include per-app success flags and millisecond timing.

C. File Management and CRUD Operations Two separate modules handle file-related work.

The first, the file operations module, handles create, read, update, append, delete, rename, and directory listing.

For creation and update intents that include a content description, the LLM generates the content directly — typing “make a Python script that reads a CSV and plots it” produces a file you can actually run.

Deletes and overwrites require explicit confirmation before executing.

Parent directories that do not exist are created automatically.

The second module, the file manager, handles search.

Its three-stage pipeline is described in Algorithm 3.

Algorithm 3 Semantic File Search Require: query q, search roots D, file type filters T 1: Stage A (type detection): extract type hints from q using keyword matching; restrict T accordingly 2: Stage B (parallel scan): for each d ∈ D spawn a thread that walks the directory tree, skipping noise directories (node_modules, .git, cache paths); collect candidate paths C; concurrently run LLM keyword extraction on q 3: Stage C (ranking): score each c ∈ C by keyword overlap with stem, filename, and path; pass top-k candidates to LLM for semantic re-ranking; return ordered list 4: return top-ranked files with open-with recommendation Each root directory is capped at 2,000 file entries, and directories like node_modules and .git are skipped outright — without those exclusions, a single npm project would exhaust the budget in one subdirectory alone.

D. AI Shell and Command Translation The /shell prefix routes input to the AI shell, which translates a plain English description into a runnable shell command.

An internal dictionary of about 40 frequent operations — disk space checks, network diagnostics, running process lists, environment variable queries — is matched first because it is faster and more reliable than asking the model.

Anything outside that set goes to the LLM with the target platform specified in the prompt, so the generated command syntax is correct for Windows, Linux, or macOS.

Before anything runs, every generated command passes through a quick safety check.

Things that trip it: recursive deletion patterns, disk format operations, kill signals aimed at system processes.

When something gets blocked, the user sees the specific reason — not a generic refusal.

A dry-run flag is also available, which prints the generated command without executing it, useful when you just want to inspect what the system would have done.

E. Workflow Automation The workflow engine treats multi-step tasks as named, persistent objects you can call up by name in any later session.

Each is represented as a directed acyclic graph where every node specifies an action type, a target list, and a group identifier.

Nodes sharing a non-zero group run concurrently; group-zero nodes are barriers — they block until all previous nodes complete.

Timing is recorded per node, which makes it easy to see after the fact where a workflow spent its time.

Users define workflows in natural language.

A sentence like “create a workflow called dev-start that opens VS Code, Chrome, and a terminal” produces a saved JSON object that can be re-executed by name in future sessions.

VI. IN-CONTEXT REINFORCEMENT LEARNING ADAPTATION We ruled out fine-tuning early.

Running gradient updates on a 3B model locally needs a GPU with headroom above what Ollama is already using, and partial fine-tuning on a narrow correction dataset risks degrading the model’s broader capabilities.

In-context learning sidesteps both problems.

It needs disk space and a similarity function.

That is it.

The RL memory component tracks three things.

A frequency map records how often each of the seven intent categories has been invoked.

A correction log stores every (incorrect intent, correct intent) pair that a user has explicitly provided.

And a user profile — a short free-text paragraph — is periodically regenerated by asking the LLM to summarise the frequency map.

The profile is rebuilt every 50 interactions.

Just before Stage 2 assembles its prompt, a quick Jaccard similarity pass runs over all stored corrections — token-set intersection over union, nothing fancier than that.

The five corrections that score highest against the current input get pulled into the prompt as examples, in the form “when the user wrote X, the right interpretation was Y.” To the model, these are indistinguishable from hand-written few-shot demonstrations.

The weights do not move.

No retraining of any kind.

Only the context changes.

Fig. 2 shows what accuracy looks like over 500 interactions.

Starting at about 67% at the 100-interaction mark, the system picks up roughly 30 percentage points by the time it reaches 400 interactions, measured against a held-out test set we did not touch during data collection.

The small upward steps scattered through the curve are profile rebuilds, each sharpening the behavioural summary slightly.

The right-hand panel is a screenshot of the actual memory files from a live session we ran — 17 total interactions logged at that point, corrections file non-empty, profile file generated.

Fig. 2.

RL Memory Adaptation.

Accuracy gain over 500 interactions with vs.

without in-context correction injection (left) and live memory file statistics (right).

VII. PARALLEL EXECUTION FRAMEWORK Every multi-target operation goes through a shared ThreadPoolExecutor.

The pool starts with eight workers at system launch and remains alive until the process exits, which avoids thread-creation overhead on individual requests.

Fig. 3 compares wall-clock times for sequential and parallel dispatch across task counts from one to eight.

For a single task, the sequential path is faster.

Thread dispatch overhead is not free, and for a single trivial operation it outweighs any benefit.

This is expected.

The thread pool is not designed to help with single tasks.

When four or more operations fire at once — say, opening four applications in one command — parallel dispatch cuts total wall time to roughly the slowest individual operation rather than the sum.

Fig. 3.

Parallel Execution Benchmark.

Wall-clock time (left) and speedup ratio (right) for sequential vs. thread-pool dispatch across 1–8 concurrent tasks.

The tasks in this benchmark are mocked — lightweight sleeps rather than real system calls.

That is why the speedup ratio dips below 1.0 at low counts.

With real workloads like process spawning, file I/O, and network requests, the crossover point where parallel wins shifts to smaller task counts.

The eight-worker default was chosen based on observed real-world usage patterns where multi-app launches of two to four targets are common.

VIII. EXPERIMENTAL EVALUATION A.

Experimental Setup Everything ran on one physical machine: consumer 8-core CPU, 16 GB RAM, an NVIDIA GPU with 4 GB VRAM.

The LLM backend was Ollama serving llama3.2:3b from its 2.0 GB quantised checkpoint.

We did not call any cloud APIs at any point during testing — every inference request went to the local Ollama instance.

The test set is 250 utterances that we labelled by hand, covering all seven intent categories.

B. Intent Classification Accuracy Four metrics are tracked in Fig.

4: accuracy, F1, precision, and recall, each across ten training epochs.

The steepest gains come in the first half; around epoch seven the curves have mostly levelled out.

Final epoch values: 91.4% accuracy, 90.1% F1, 89.2% precision, 91.0% recall.

Training and validation curves tracked each other fairly closely throughout — no obvious sign of the model memorising the training set rather than learning from it.

C. Model Perplexity Fig.

5 plots train, validation, and test perplexity.

At epoch 1 all three are in the 143–165 range.

At epoch 10 they land at 23, 37, and 38 respectively.

Validation and test curves stay very close to each other, which indicates the model is generalising Fig.

4. Intent Classification Performance.

Accuracy, F1, precision, and recall across 10 epochs; final values 91.4% / 90.1% at epoch 10. rather than memorising.

We did not use early stopping; the curves did not require it.

Fig. 5.

Model Perplexity Curves.

Train/val/test perplexity converging from ~150 to 23/37/38 over 10 epochs with no early stopping.

D. Training Loss Curves Fig.

6 separates the two loss components into side-by-side panels.

Training cross-entropy went from 2.85 down to 0.63; validation moved from 3.12 to 1.07.

That train-validation gap narrowed as training progressed, which is a decent sign.

L2 loss dropped from 0.31 to 0.17 without reversing at any epoch.

The decreasing regularisation alongside better accuracy is consistent with the model finding more compact representations of the intent structure — though we acknowledge that interpretation is partly speculative without a deeper ablation.

Fig. 6.

Training Loss Curves.

Cross-entropy (left) and L2 regularisation loss (right) falling monotonically over 10 epochs.

E. Comparative Evaluation Against Baselines We compared three configurations: the full hybrid system, Stage 1 alone (rule-only), and Stage 2 alone with no regex front-end.

Fig. 7 plots five numbers for each — intent accuracy, app launch success rate, workflow completion rate, file operation success rate, and URL routing precision.

The hybrid came out ahead on all five.

App launch is where the margin is widest: hybrid hit 100%, rule-only 76%, LLM-only 67%.

In practice, neither component alone handles the range of application names that people actually type, and the gap shows it.

Intent accuracy is harder to interpret at face value.

The rule-only 53.6% figure is calculated only over inputs the rules actually matched — it counts nothing the rule engine refused to handle.

AIOS at 30.0% and LLM-only at 24.6% are both measured across the full test set, including everything the rule engine would have dropped silently.

Fig. 7.

Baseline Comparison.

AIOS (Hybrid) vs.

Rule-only and LLM-only across five metrics; AIOS leads on all five.

F. Per-Tool Accuracy by Invocation Complexity Fig.

8 breaks things down by module and by how many tool calls the intent required.

Single-invocation accuracy ranged from 84% on File Ops to a perfect 100% on App Launcher.

Once a single intent required four or more module calls, that range dropped to 70–84%.

App Launcher held up the best across complexity levels — executable path resolution is essentially a lookup, and lookups succeed or fail with no ambiguity.

File Ops degraded the most, which makes intuitive sense: the same natural language phrase can reasonably mean different file operations depending on what the user actually had in mind, and the model gets it wrong more often as the complexity grows.

G. Intent Classification Confusion Matrix Fig.

9 is the normalised confusion matrix across all seven categories under live LLM evaluation.

Diagonal entries sit at 30% across the board.

Off-diagonal mass is distributed fairly evenly, roughly 12% per pair, with no specific category combination jumping out as a primary confusion source.

We actually found this more reassuring than a matrix with concentrated off-diagonal entries would have been.

A uniform spread tells you Fig.

8. Per-Tool Accuracy.

Exact-match scores per module at 1-, 2-, 3-, and 4+-invocation complexity.

the errors are not coming from a structural problem with how two specific categories overlap in the model’s representation — more labelled data would likely help all classes roughly equally, rather than requiring a targeted architectural fix.

Fig. 9.

Intent Confusion Matrix.

Normalised recall (%) across all seven intent classes; uniform off-diagonal entries indicate no systematic confusion.

H. System Efficiency and Resource Utilisation Live latency and resource numbers are in Table I; Fig.

10 shows the same data as a normalised heat map to make the relative magnitudes easier to read at a glance.

The number that stands out is the latency gap between the two stages.

Stage 1 tops out at 0.20 ms — essentially below timing precision on most machines.

Stage 2 starts at 6,073 ms on its fastest run.

That is roughly six orders of magnitude.

Every time Stage 1 matches an input, a six-to-fifteen-second inference call simply does not happen — and that observation is the core argument for the hybrid design.

On the resource side, GPU memory averaged 2.68 GB across our test sessions, which left meaningful headroom on the 4 GB card.

CPU sat

TABLE I AIOS EFFICIENCY METRICS: LATENCY AND RESOURCE UTILISATION (LIVE)			
Metric	Min	Avg	Max
Tool Routing Latency (ms)	0.00	0.00	0.20
App Normalisation Latency (ms)	0.00	0.00	0.00
LLM Intent Parse Latency (ms)	6073	7994	15467
Speech Recognition Latency (ms)	150	380	750
Speech Synthesis Latency (ms)	0.00	0.00	0.00
CPU Usage (%)	7.09	20.0	37.0
RAM Usage (GB)	7.27	13.0	18.0
GPU Memory (GB)	1.47	2.68	3.67

at 20% average, so the system was not squeezing out other work on the machine.

Fig. 10.

Efficiency Heat Map.

Min/Avg/Max normalised latency and resource usage per module; LLM inference dominates all other costs.

I. Live System Demonstration What follows are eight screenshots from a session we actually ran on a Windows 11 machine with Ollama hosting llama3.2:3b locally.

None of them were edited or staged after capture.

Fig. 11 captures startup: Ollama connects, the banner comes up.

The first input we typed was just “hi” — Stage 1 matched it to a greeting and responded before the LLM was involved.

We then asked it to make hello.py with a hello world function.

The model wrote four lines of valid Python to disk, and a read hello.py command right after confirmed the file was actually there.

Fig. 12: the AI shell in use.

The input "show installed python version" is translated to python --version, which returns Python 3.11.8.

Working directory is tracked and visible throughout.

Fig. 13: file update.

“Update hello.py to add error handling” causes the LLM to rewrite the file with a try/except block wrapped around the original function.

The updated five-line content appears inline.

Fig. 11.

System Startup and File Creation.

AIOS startup (Ollama llama3.2:3b), Stage 1 greeting, and AI-assisted Python file creation with live 4-line content preview.

Fig. 12.

AI Shell Demo.

/shell: natural language translated to python --version, returning Python 3.11.8 with working-directory tracking.

Fig. 14: safe deletion.

“Delete hello.py” triggers a pre- execution report (130 bytes, file exists) and then a y/N prompt.

The file is only removed after the user confirms.

Fig. 15: /kill notepad finds Notepad.exe at PID 17688, consuming 153 MB.

The user confirms the PID and the process is terminated.

The same command aimed at a system-critical process name would be blocked at the protection list check.

Fig. 16: the /ps process view.

Header shows CPU at 23.9%,

12.5 of 15.7 GB RAM used, 355 GB disk free. Below that, a colour-coded table lists the top 14 processes sorted by resident Fig.

13. AI File Update.

LLM rewrites hello.py adding a try/except block; the 5-line updated content is shown inline.

Fig. 14.

Safe File Deletion.

Byte-level metadata (130 bytes) and explicit y/N confirmation before the destructive operation is executed. memory, with PID, name, CPU percentage, RAM in MB, and status for each.

Fig. 17 packs three different things into one capture.

Chrome and Terminal both launched from a single command — Chrome in 35 ms, Terminal in 71 ms, total wall time 75 ms because they ran simultaneously.

A YouTube song query was passed off to a search URL in 20 ms.

A file search for health- related content across four separate directory roots came back with eight results, ranked by relevance.

Fig. 18 shows the Cognitive Pipeline Dashboard.

The currently active step is highlighted across the execution chain: User Input→ LLM Engine→ Intent

Parser→ Syscall Planner → Kernel Dispatcher → Process Manager → OS Response Buffer → Action Complete.

On the right is a live event log showing WARN, CRIT, and OK entries from the PROC, RAM, and SYS

subsystems, alongside four resource gauges.

Fig. 15.

Process Kill Demo.

/kill notepad: PID 17688 identified (153 MB), confirmation prompt, and successful Notepad.exe termination.

Fig. 16.

Process Manager (/ps).

Live system stats (CPU 23.9%, RAM 12.5/15.7 GB) and colour-coded top-14 process table ranked by resident memory (ollama.exe leads at 746 MB).

Fig. 17.

Parallel Launch and File Search.

Parallel app launch (Chrome + Terminal, 75 ms total), YouTube URL routing (20 ms), and semantic file search with ranked multi-root results.

Fig. 18.

Cognitive Pipeline Dashboard.

Live execution path, resource gauges (CPU, RAM, Disk, Network), and WARN/CRIT/OK event log across PROC, RAM, and SYS subsystems.

IX. DISCUSSION A.

Strengths of the Hybrid Pipeline Both components fail, just in different directions.

Rules break the moment something falls outside their vocabulary — a slightly different phrasing, an app name that was never written in, and the match does not fire.

The LLM is the opposite problem: it handles unfamiliar inputs fine, but it adds latency to everything including commands that never need model inference, and it produces plausible-looking but wrong values like process IDs and file paths often enough to be a real issue.

Using both together sidesteps the worst of each — fast and deterministic for well-understood inputs, flexible for everything else.

Tiered dispatch architectures are not new.

The specific combination we were trying to make work — fully local, no pre-approved command vocabulary, natural language OS interface — is less common.

B. Platform Agnosticism Platform compatibility was a priority from the start, though we should be honest that most of our actual testing happened on Windows.

The abstract pieces are genuinely portable: psutil for processes, pathlib for filesystem paths, and an application resolver that chains through four strategies covering most installation patterns on both POSIX and Windows.

What is not portable is the application alias dictionary.

But that is a configuration file, not code — swapping it for a different platform does not require touching anything else.

C. Limitations and Future Work The most visible issue is Stage 2 latency.

Eight seconds average is fine for a slow deliberate query like “find all database-related Python files and summarise them” — that warrants the wait.

For “open Spotify,” it is obviously not acceptable.

Stage 1 handles that case, which is exactly why the hybrid exists.

But Stage 1’s coverage is bounded, and anything outside it hits the model.

Getting Stage 2 under two seconds would change the feel of the system significantly; smaller quantised models around 1B parameters and speculative decoding both look worth trying.

Recall numbers from live evaluation are also low, roughly 30% per class.

We do not think this is an architectural problem.

It is a data problem.

250 utterances split across seven categories leaves each category thin.

A few thousand labelled examples would almost certainly help, without requiring any change to the system itself.

One more practical issue: the file search module rescans the filesystem from scratch on every query.

For most home directories this does not matter.

On a machine with a large, deeply nested project tree it would become slow.

An incremental index updated on filesystem events rather than rebuilt per query is the obvious fix — we just have not built it yet.

X. CONCLUSION When we started building AIOS, the question we kept coming back to was whether a 3B model running locally, with no cloud APIs and no kernel patches, could actually serve as a natural language interface to an OS.

Not a limited demo.

Something that opens applications, manages running processes, reads and writes files, executes shell commands, and chains those together in repeatable multi-step workflows.

The answer we arrived at is roughly yes, but with real caveats.

The model alone is too slow.

Eight seconds average response time is fine for some requests and completely wrong for others.

What made the system usable in practice is the rule engine out front.

Inputs that appear frequently and have predictable structure resolve in under a millisecond; the LLM only runs when the rule set genuinely cannot handle what was typed.

That split is what separates a technically interesting result from something that works in day-to-day use.

The correction loop worked better than we expected.

User corrections fed back as few-shot examples produced measurable accuracy gains over time, without touching the model's weights at any point.

Numbers: 91.4% intent accuracy, 90.1% F1, ahead of both the rule-only and LLM-only baselines on every metric we measured.

We think this is a solid foundation.

Getting Stage 2 latency below two seconds and training on more labelled data are the two most direct paths forward.

REFERENCES

- [1] W. Liu, G. Zhuang, X. Liu, S. Hu, R. He, and Y. Wang, "How do we move towards true artificial intelligence," in Proc. IEEE 23rd Int. Conf. High Performance Computing & Communications (HPCC/DSS/SmartCity/DependSys), 2021.
- [2] M. Masrom, L. Krisnan, Y. Yahya, and M. K. Kaundan, "Patient attitude on the application of artificial intelligence in diabetes care," in Proc. 5th Int. Conf. Artificial Intelligence and Data Sciences (AiDAS), 2024.
- [3] M. Song and X. Chen, "Construction of enterprise business management analysis framework in the development of artificial intelligence," in Proc. Int. Conf. Computer Information Science and Artificial Intelligence (CISAI), 2021.
- [4] G. Chen and Q. Yuan, "Application and existing problems of computer network technology in the field of artificial intelligence," in Proc. 2nd Int. Conf. Artificial Intelligence and Computer Engineering (ICAICE), 2021.
- [5] C. Zhu and Y. Guan, "The risks and countermeasures of accounting artificial intelligence," in Proc. 3rd Int. Conf. Electronic Communication and Artificial Intelligence (IWECAI), 2022.
- [6] J. Harika, P. Baleeshwar, K. Navya, and H. Shanmugasundaram, "A review on artificial intelligence with deep human reasoning," in Proc. Int. Conf. Applied Artificial Intelligence and Computing (ICAAIC), 2022.
- [7] X. Niu and L. Zhang, "Research on the application of artificial intelligence in hotels," in Proc. 3rd Int. Conf. Artificial Intelligence and Computer Information Technology (AICIT), 2024.
- [8] R. Sun, "Analysis on the application of artificial intelligence in marketing," in Proc. Int. Conf. Computer Information Science and Artificial Intelligence (CISAI), 2021.

- [9] X. Ma, "Application of artificial intelligence in computer network technology," in Proc. 2nd Int. Conf. Artificial Intelligence and Autonomous Robot Systems (AIARS), 2023.
- [10] Z. Qian, "Applications, risks and countermeasures of artificial intelligence in education," in Proc. 2nd Int. Conf. Artificial Intelligence and Education (ICAIE), 2021.
- [11] I. Yadav, V. Shekhawat, K. Gautam, G. K. Soni, and R. Yadav, "Artificial intelligence for cybersecurity: Emerging techniques, challenges, and future trends," in Proc. 3rd Int. Conf. Sustainable Computing and Data Communication Systems (ICSCDS), 2025.
- [12] S. Kacimi and F. Bennouna, "Responsible artificial intelligence in project management," in Proc. 4th Int. Conf. Embedded Systems and Artificial Intelligence (ESAI), 2025.
- [13] Y. Jia, "Analysis of the impact of artificial intelligence on electricity consumption," in Proc. 3rd Int. Conf. Artificial Intelligence, Internet of Things and Cloud Computing Technology (AIoTC), 2024.
- [14] Z. Qian, "Exploration and reflection of legal artificial intelligence in Japan," in Proc. 8th Asian Conf. Artificial Intelligence Technology (ACAIT), 2024.
- [15] C. Tang, Z. Wang, X. Sima, and L. Zhang, "Research on artificial intelligence algorithm and its application in games," in Proc. 2nd Int. Conf. Artificial Intelligence and Advanced Manufacture (AIAM), 2020.
- [16] N. Y. Fares and M. Jammal, "Decoding justice: The synergy of artificial intelligence and machine learning in the legal landscape," in Proc. IEEE Global Conf. Artificial Intelligence and Internet of Things (GCAIoT), 2024.
- [17] B. Tural, Z. Örpek, and Z. Destan, "Ethics in the age of artificial intelligence," in Proc. 3rd Cognitive Models and Artificial Intelligence Conf. (AICCONF), 2025.
- [18] L. Xiaojun, Z. Xiande, Z. Kexi, D. Zhenli, Z. Kai, and W. Xi, "Study on the application fields and development prospects of artificial intelligence," in Proc. 2nd Int. Conf. Artificial Intelligence and Education (ICAIE), 2021.
- [19] G. Liu, H. Wan, and L. Zhang, "Application of artificial intelligence in computer network technology in the era of big data," in Proc. 2nd Int. Seminar Artificial Intelligence, Networking and Information Technology (AINIT), 2021.
- [20] S. V. Khara, G. D. Tivari, S. Patel, J. Nathvani, R. Amrutiya, and D. Chavda, "Generative artificial intelligence for intelligent decision support in organizational management: A data-driven framework," in Proc. 3rd Int. Conf. Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE), 2026.